

GraphManager: Cost-Aware Graph-Guided Retrieval for Issue Localization in Software Repositories

Author Name(s)

February 22, 2026

Abstract

Accurate file localization is a critical first step in automated software issue resolution, yet the dominant retrieval approach—dense embedding of code chunks—is both expensive to set up and structurally unaware of repository dependencies. We present GraphManager, a retrieval framework that operates over a graph world model derived from the repository’s abstract syntax tree, encoding files, classes, and functions with typed structural edges (imports, calls, inheritance). The framework supports seven retrieval methods spanning deterministic graph scoring, LLM-guided graph navigation, and standard RAG baselines, enabling direct comparison of retrieval quality against token cost. We introduce a dual-track evaluation protocol that separately reports strict commit-fidelity and same-snapshot amortized settings, preventing conflation of retrieval quality with setup cost structure. Evaluated on SWE-bench issues across multiple Python repositories, graph-guided methods achieve competitive file-level F1 while requiring substantially fewer total tokens than RAG baselines, with the graph index setup costing approximately $3\times$ fewer embedding tokens than dense code chunk indexing. The deterministic graph scorer provides a strong zero-LLM-runtime baseline, while LLM-guided graph navigation further improves recall on structurally complex repositories. A patching pilot on 100 SWE-bench Verified instances across 9 repositories shows directional evidence that the cost advantage extends end-to-end: GM-progressive resolves 43% of instances at approximately $4.4\times$ lower method-accounted cost per resolved issue than RAG-progressive. All retrieval experiments use manifest-pinned issue sets with paired per-issue deltas and bootstrap confidence intervals.

1 Introduction

Automated resolution of software issues—from bug reports to feature requests—has emerged as a central application of large language models (LLMs) in software engineering [7, 18, 17, 19]. A critical first step in any issue resolution pipeline is *file localization*: identifying which files in a repository are relevant to the issue at hand. The quality of this retrieval step directly constrains downstream patch

generation, as irrelevant or missing files lead to incorrect or incomplete fixes.

The dominant approach to file localization is dense retrieval, typically implemented as retrieval-augmented generation (RAG) [9]: code is chunked, embedded into a vector index, and the top- K chunks most similar to the issue description are retrieved. While effective, this approach has two structural limitations. First, it is *expensive to set up*: embedding an entire codebase requires substantial API calls, and the resulting index must be rebuilt whenever the repository state changes. Second, it is *structurally unaware*: embedding similarity captures lexical and semantic relatedness but does not encode the dependency structure—imports, function calls, class inheritance—that developers routinely use to navigate code.

We propose *GraphManager*, a retrieval framework that operates over a graph world model derived from the repository’s abstract syntax tree (AST). The graph encodes files, classes, and functions as nodes, with typed edges for structural relations (imports, calls, inheritance, containment). A FAISS vector index over node metadata provides semantic entry points into the graph, while graph traversal supplies structural context that flat retrieval misses.

The framework supports a spectrum of retrieval policies: from a fully deterministic graph scorer that uses no LLM calls at query time, to LLM-guided interactive agents that navigate the graph through function-calling tools. This range enables direct comparison of retrieval quality against cost, and isolates the contribution of graph structure from that of LLM reasoning.

A key design principle is *cost awareness*. We decompose total token cost into three components—setup embedding, runtime LLM, and query embedding—and report all three for every method. This accounting reveals a structural advantage of graph-based retrieval: because the graph index is built from compact structural metadata rather than full code chunks, its setup cost is substantially lower than that of a dense code embedding index. When the same graph serves multiple retrieval tasks against a single repository snapshot, this setup cost is amortized further.

To evaluate this tradeoff rigorously, we introduce a dual-track evaluation protocol:

1. **Strict commit fidelity**: each issue is evaluated at its own `base_commit`, requiring per-commit index re-

builds and providing a conservative cost estimate.

2. **Same-snapshot amortized:** all issues for a repository are evaluated against a single fixed snapshot, measuring the cost benefit of world-model reuse.

We evaluate seven retrieval methods—including deterministic graph scoring, LLM-guided graph navigation, LLM-guided RAG, and non-LLM RAG baselines—on issues drawn from SWE-bench [7] across multiple Python repositories.

Contributions.

1. A graph-guided retrieval framework with seven method variants, including a deterministic multi-component scorer that requires no LLM calls at query time.
2. An amortization-aware dual-track evaluation protocol that separately reports strict commit-fidelity and same-snapshot reuse settings, preventing conflation of retrieval quality with cost structure.
3. A reproducible cost-quality analysis on SWE-bench showing that graph-based methods achieve competitive file-level F1 at lower total token cost than RAG baselines.
4. A patching pilot on 100 SWE-bench Verified instances across 9 repositories providing directional evidence that the retrieval cost advantage extends end-to-end to issue resolution.

2 Related Work

2.1 Repository-Level Benchmarks

Evaluating code understanding systems at the repository level requires benchmarks that capture cross-file dependencies and real-world issue complexity. SWE-bench [7] introduced a large-scale benchmark of real GitHub issues with executable patch validation, establishing the standard for evaluating automated issue resolution. SWE-PolyBench [15] extended this paradigm to multiple programming languages and introduced retrieval-specific evaluation modes, including node-level metrics that align well with graph-based retrieval.

Complementary benchmarks address adjacent tasks: RepoBench [10] evaluates repository-level next-line code completion under cross-file context, while RepoExec [6] focuses on executable function completion with varying context budgets. We use SWE-bench as our primary evaluation dataset and SWE-PolyBench for cross-benchmark validation, as both provide the issue-level localization task that our retrieval methods target.

2.2 Graph-Based Code Retrieval

Several recent systems exploit repository structure—dependency graphs, call graphs, and AST-derived relations—to improve code retrieval beyond flat embedding search.

RepoGraph [13] formalizes the repository graph as a first-class object for AI-driven software engineering, constructing file-level dependency graphs and demonstrating that hop-structured context assembly improves downstream code generation. RepoHyper [14] introduces a search-expand-refine pattern on semantic graphs, where an initial retrieval step is followed by graph-guided expansion with explicit budget control. GraphCoder [11] applies graph-based coarse-to-fine reranking to repository-level code completion, using code context graphs to improve retrieval precision.

RepoScope [12] addresses call chain-aware multi-view retrieval, constructing static call chains and using them as structured context views for code generation. Chinthareddy [2] compares AST-derived graphs against LLM-extracted knowledge graphs for code retrieval, finding that deterministic AST parsing provides more reliable structure than LLM-generated alternatives.

Our graph construction follows the deterministic AST-first approach advocated by Chinthareddy [2], with modular import/call/type resolution inspired by the open-source `code-graph-rag` project [16]. We extend this foundation with confidence-tiered call edges, a multi-component deterministic scoring formula, and explicit cost accounting that prior graph retrieval work does not report.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [9] augments language model generation with retrieved non-parametric evidence, and has become the dominant paradigm for grounding LLM outputs in code. Standard code RAG systems embed code chunks into a dense vector index and retrieve the top- K most similar chunks given a query.

GraphRAG [3] demonstrated that augmenting flat retrieval with graph-structured summaries improves query-focused summarization for general text. In the code domain, graph structure offers a natural complement to dense retrieval: import and call edges encode dependency paths that embedding similarity alone cannot capture. Our system uses vector similarity as the *entry point* into the graph, combining the recall advantages of dense retrieval with the structural precision of graph navigation.

2.4 Automated Issue Resolution Systems

Automated systems for resolving software issues require accurate file localization as a critical first step. SWE-agent [18] provides an agent-computer interface for navigating and editing repositories, relying on interactive exploration to locate relevant files. Agentless [17] takes a contrasting

approach, decomposing issue resolution into localization and repair phases without persistent agent state, achieving competitive results with simpler orchestration. AutoCodeRover [19] combines code search with AST-aware context retrieval for autonomous program improvement.

These systems highlight that file localization quality directly impacts downstream patch success, yet most evaluate localization only implicitly through end-to-end patch metrics. Our work isolates the retrieval step and evaluates it with explicit quality and cost metrics, providing a focused comparison of retrieval strategies that can inform any downstream system.

2.5 Positioning

Our work sits at the intersection of graph-based code retrieval and cost-aware evaluation for issue localization. Unlike prior graph retrieval systems that report only quality metrics, we provide explicit three-component cost accounting (setup, LLM runtime, query embedding) and a dual-track evaluation protocol that separates strict commit-fidelity from amortized same-snapshot reuse. Unlike RAG-only systems, we demonstrate that structural graph navigation can achieve competitive retrieval quality at substantially lower total token cost.

3 Method

We present a retrieval framework that operates over a structural representation of a software repository—a *graph world model*—rather than over flat text embeddings alone. The framework supports multiple retrieval policies, ranging from a fully deterministic graph scorer to LLM-guided interactive search, all sharing the same underlying graph and index infrastructure. This section describes the graph construction (§3.1), the deterministic retrieval algorithm (§3.2), the LLM-guided retrieval policies (§3.3), and the cost model used for comparison (§3.5).

3.1 Graph World Model

Given a repository snapshot at a specific commit, we construct a directed, typed knowledge graph $G = (V, E)$ using static analysis of the Python source tree.

Nodes. The node set V contains three entity types:

- **File nodes:** one per Python source file within the configured source prefixes.
- **Class nodes:** one per class definition, storing the class name, docstring, and base classes.
- **Function nodes:** one per function or method definition, storing the function name, signature, and docstring.

Crucially, node attributes include only structural metadata—names, signatures, and docstrings—but *not* implementation bodies. This keeps the graph compact and avoids leaking solution-relevant code into the retrieval signal.

Edges. The edge set E contains five typed relations:

- **DEFINES:** file $\rightarrow$ class or function defined in that file.
- **CONTAINS:** class $\rightarrow$ method defined within that class.
- **IMPORTS:** file $\rightarrow$ file, derived from import statements.
- **CALLS:** function $\rightarrow$ function, resolved by static call analysis.
- **INHERITS:** class $\rightarrow$ class, derived from base class declarations.

Call resolution with confidence tiers. Call edges are the most challenging to resolve statically in a dynamic language like Python. We decompose call resolution into three modular components: an **ImportResolver** that builds a per-file symbol table from import statements (including aliases, relative imports, and star imports), a **TypeInferenceEngine** that propagates type hints and simple assignment-based types, and a **CallResolver** that matches call sites to target functions using the resolved symbol context.

Each call edge is tagged with a confidence tier:

- **High:** target resolved via explicit import or type annotation.
- **Medium:** target matched by name within the same module.
- **Low:** target matched by global suffix or name-only fallback.

Low-confidence edges can be optionally excluded via an ablation switch, enabling controlled experiments on graph precision versus recall.

Semantic index. We build a FAISS [8] flat inner-product index over vector embeddings of node text (concatenation of name, signature, and docstring). Embeddings are produced by **gemini-embedding-001** [4]. This index serves as the *semantic entry point* into the graph: given an issue description, we retrieve the top- K most similar nodes and use them as seeds for graph-based expansion.

Parsing infrastructure. AST parsing uses **tree-sitter** [1] for language-agnostic syntax tree construction. The graph is stored and queried using NetworkX [5].

3.2 Deterministic Graph Retrieval

The deterministic retriever (`gm_deterministic`) is a zero-LLM-runtime method that produces a ranked file list from the graph without any language model calls at query time.

Algorithm overview. Given an issue description q :

1. **Seed retrieval:** query the FAISS index with q to obtain the top- K_{seed} nodes (default $K_{\text{seed}} = 8$), each with a normalized semantic similarity score.
2. **Bounded graph expansion:** from each seed node, run bidirectional BFS with maximum depth D (default 2) and maximum neighbors per node N (default 12), traversing all edge types. Record each discovered path with its source seed, edge types, and edge confidence.
3. **File evidence aggregation:** map each visited node to its containing file and aggregate five evidence components per file.
4. **Scoring and ranking:** compute a weighted score per file and return the top- K_{output} files.

Scoring formula. For each candidate file f , the score is:

$$s(f) = w_{\text{sem}} \cdot S_{\text{sem}}(f) + w_{\text{graph}} \cdot S_{\text{graph}}(f) + w_{\text{conf}} \cdot S_{\text{conf}}(f) + w_{\text{hint}} \cdot S_{\text{hint}}(f) - w_{\text{pen}} \cdot S_{\text{pen}}(f) \quad (1)$$

where:

- $S_{\text{sem}}(f)$: maximum seed similarity among nodes in f .
- $S_{\text{graph}}(f)$: graph support strength, based on the count and diversity of independent paths reaching f .
- $S_{\text{conf}}(f)$: confidence quality of supporting paths, weighting high-confidence edges at 1.0, medium at 0.6, and low at 0.2.
- $S_{\text{hint}}(f)$: lexical overlap between the issue text and the file or module name.
- $S_{\text{pen}}(f)$: hub penalty for files reachable only through low-confidence or shallow-path evidence.

Default coefficients are $w_{\text{sem}} = 0.35$, $w_{\text{graph}} = 0.30$, $w_{\text{conf}} = 0.20$, $w_{\text{hint}} = 0.10$, $w_{\text{pen}} = 0.05$.

3.3 LLM-Guided Retrieval Policies

In addition to the deterministic retriever, we evaluate LLM-guided retrieval via two agent families that use `gemini-2.0-flash` [4] as the language model backend with function calling.

Graph manager agent. The `ManagerAgent` operates over the graph world model through three tools:

- `search_nodes(query, top_k)`: semantic search over the graph node index.
- `get_neighbors(node_id)`: retrieve typed neighbors of a node in the graph.
- `get_file_summary(file_path)`: retrieve structural metadata for a file (defined classes, functions, imports).

The agent is prompted with the issue description and iteratively navigates the graph to identify relevant files, constrained by a maximum turn budget.

RAG agent. The `RAGAgent` operates over a standard dense vector index built from code chunks (either function-level or fixed-size). It uses a single tool:

- `search_codebase(query, top_k)`: dense retrieval over embedded code chunks.

Retrieval modes. Each agent family supports two modes:

- **Baseline:** the agent receives the full issue text and produces a file list in a single interaction pass.
- **Progressive:** the agent receives a cleaned and optionally path-redacted version of the issue text, uses compact tool payloads (identifiers and scores only), and applies evidence-weighted ranking with confirmed-file gating when confirmations exist.

3.4 Methods Compared

Table 1 summarizes the seven retrieval methods evaluated.

3.5 Cost Model

We decompose the total token cost into setup and runtime components to enable fair comparison across methods with different index construction costs.

For method m over N issues:

$$C_m^{\text{total}} = C_m^{\text{setup}} + \sum_{i=1}^N (C_{m,i}^{\text{llm}} + C_{m,i}^{\text{qemb}}) \quad (2)$$

where C_m^{setup} is the one-time embedding cost for index construction (graph node embeddings or code chunk embeddings), $C_{m,i}^{\text{llm}}$ is the LLM token usage for issue i , and $C_{m,i}^{\text{qemb}}$ is the query embedding cost for issue i .

The per-issue amortized cost is:

$$C_m^{\text{amortized}} = \frac{C_m^{\text{total}}}{N} \quad (3)$$

Method	Description	Index	LLM at query	Graph
<code>gm_deterministic</code>	Deterministic graph scorer	Graph + FAISS	No	Yes
<code>gm_progressive</code>	Graph manager, progressive mode	Graph + FAISS	Yes	Yes
<code>gm_baseline</code>	Graph manager, baseline mode	Graph + FAISS	Yes	Yes
<code>rag_progressive</code>	RAG agent, progressive mode	Chunk FAISS	Yes	No
<code>rag_baseline</code>	RAG agent, baseline mode	Chunk FAISS	Yes	No
<code>raw_rag_function</code>	Top- K function chunks, no LLM	Chunk FAISS	No	No
<code>raw_rag_fixed</code>	Top- K fixed chunks, no LLM	Chunk FAISS	No	No

Table 1: Summary of retrieval methods. “Graph” indicates whether the method uses the AST-derived knowledge graph. “LLM at query” indicates whether language model calls are made during per-issue retrieval.

Break-even analysis. Given two methods m and b where m has lower setup cost but higher per-issue runtime cost, the break-even task count is:

$$N_{\text{break-even}} = \frac{C_m^{\text{setup}} - C_b^{\text{setup}}}{C_b^{\text{runtime}} - C_m^{\text{runtime}}} \quad (4)$$

where $C^{\text{runtime}} = C^{\text{llm}} + C^{\text{qemb}}$ is the mean per-issue runtime cost. When the denominator is positive, $N_{\text{break-even}}$ gives the number of tasks at which method m becomes cheaper than b in total cost.

4 Experimental Setup

4.1 Datasets

We draw evaluation instances from two benchmark families:

SWE-bench [7] provides real GitHub issues paired with developer-written patches. Each instance specifies a repository, a `base_commit` at which the issue exists, and a gold patch from which we extract gold file labels. The full dataset contains 2,294 test instances, 225 dev instances, and 19,008 training instances across 12 repositories. We use the test split for evaluation.

SWE-PolyBench [15] extends repository-level evaluation to multiple programming languages (Java, JavaScript, TypeScript, Python) and provides both retrieval-metric and execution-based evaluation modes. The Verified subset contains 382 instances. We use the Python subset for cross-benchmark validation.

4.2 Repository Selection

We evaluate on seven repositories spanning four application domains, drawn from both SWE-bench and SWE-PolyBench:

The selection covers both small, well-structured libraries (Flask, Requests) and larger, more complex codebases (Keras, LangChain), providing diversity in graph structure and retrieval difficulty.

4.3 Issue Sampling and Manifest Pinning

For each repository, we select issues using a deterministic protocol:

1. Sweep all available splits (test, dev, train) and collect instances matching the target repository.
2. Deduplicate by `instance_id`.
3. Select the first n instances in dataset order.
4. Pin the resulting instance IDs in a YAML manifest with a unique `issue_set_id`.

The manifest is version-controlled and used for all subsequent runs, ensuring that different methods and repeat runs are evaluated on exactly the same issue set. Example: `issues_swebench_requests_v2_10`.

4.4 Evaluation Tracks

We report results on two parallel tracks to separate retrieval quality from amortization effects:

Track 1: Strict commit fidelity. Each issue is evaluated at its own `base_commit`. The repository is checked out and the graph/index is rebuilt for each unique commit. This is the “realistic conservative” setting: it measures retrieval quality without any amortization benefit. Issues are grouped by commit to avoid redundant rebuilds when multiple issues share a commit, but in practice the commit repeat ratio is low (often 0%) for $n = 10$ samples.

Track 2: Same-snapshot amortized. All issues for a given repository are evaluated against a single fixed snapshot (a pinned commit hash). The graph and index are built once and reused for all issues. This measures the “world-model reuse” scenario where a pre-built structural representation serves multiple retrieval tasks. The `commit_repeat_ratio` and `cache_hit_rate` are reported to quantify the amortization opportunity.

Results from the two tracks are always reported separately and never merged into a single aggregate.

Repository	Domain	Benchmark	Source prefix	n
pallets/flask	Web framework	SWE-bench	src/flask	10
psf/requests	Networking	SWE-bench	requests	10
pytest-dev/pytest	Testing/tooling	SWE-bench	src/_pytest	10
tiangolo/fastapi	Service framework	SWE-bench	fastapi	10
yt-dlp/yt-dlp	App/CLI	SWE-PolyBench	yt_dlp	5
langchain-ai/langchain	Service framework	SWE-PolyBench	libs/langchain	10
keras-team/keras	Library	SWE-PolyBench	keras	10

Table 2: Repositories in the evaluation matrix. n = number of issues per repository per track. Source prefix restricts graph construction and retrieval to relevant subdirectories.

4.5 Metrics

Retrieval quality. We compute file-level precision, recall, and F1 per issue:

$$P_i = \frac{|\hat{F}_i \cap F_i^*|}{|\hat{F}_i|}, \quad R_i = \frac{|\hat{F}_i \cap F_i^*|}{|F_i^*|}, \quad F1_i = \frac{2P_iR_i}{P_i + R_i} \quad (5)$$

where $\hat{F}_i$ is the set of files retrieved by the method and F_i^* is the gold set extracted from patch diffs. Issues that produce errors (API failures, timeouts) receive $F1 = 0$ to avoid survivorship bias. We report mean values across issues per method.

Cost metrics. For each method, we report:

- **Setup embedding tokens** (C^{setup}): embedding tokens consumed during index construction.
- **Runtime LLM tokens** (C^{llm}): language model tokens consumed during per-issue retrieval.
- **Query embedding tokens** (C^{qemb}): embedding tokens consumed during per-issue retrieval.
- **Total cost tokens:** $C^{\text{setup}} + \sum_i (C_i^{\text{llm}} + C_i^{\text{qemb}})$.

Amortization descriptors.

- **commit_repeat_ratio** = $1 - \frac{\text{unique base commits}}{n}$
- **cache_hit_rate** = $\frac{\text{reused index builds}}{n}$

4.6 Statistical Protocol

To account for LLM stochasticity, each experiment cell is repeated at least 3 times with identical configurations.

Paired analysis. For each issue i , we compute per-issue deltas between method pairs:

$$\delta_i = F1_i^{(\text{method A})} - F1_i^{(\text{method B})}$$

We report the mean delta with a bootstrap 95% confidence interval (10,000 resamples). A comparison is considered significant when the CI excludes zero.

Readiness gates. Results are promoted to the manuscript only when:

1. At least 3 repeat runs are completed.
2. Bootstrap CIs are computed for all primary comparisons.
3. The error rate is below 20% across methods.

4.7 Infrastructure

- **Retrieval language model:** gemini-3-flash-preview for all LLM-guided retrieval agents.
- **Embedding model:** gemini-embedding-001 for all vector embeddings.
- **Vector index:** FAISS [8] flat inner-product index.
- **AST parsing:** tree-sitter [1] with the Python grammar.
- **Graph library:** NetworkX [5].
- **Agent turn budget:** up to 8 turns per issue for both graph and RAG agents.

4.8 Reproducibility

All experiment configurations are specified in versioned YAML manifests (e.g., `experiments_matrix_v2.yaml`) with pinned instance IDs and snapshot commits. Run outputs are archived with timestamps, and report-ready numbers are sourced from frozen artifact bundles containing raw results, aggregated summaries, and manifest metadata. The full evaluation pipeline is executable via `python run_suite.py experiments_matrix_v2.yaml`.

4.9 Patching Pilot Setup

To validate that retrieval quality translates to end-to-end issue resolution, we conduct a patching pilot using the SWE-bench Verified subset. We draw a frozen split of $N = 100$ instances across 9 repositories (astropy, matplotlib, Flask, requests, xarray,

pytest, scikit-learn, sphinx-doc, sympy), stratified by per-repository Verified instance count. Instance IDs are pinned in a versioned manifest prior to any runs (`patch_manifests/verified_n100_split_v1.json`).

We evaluate four methods:

- **Oracle:** gold patch files fed directly to the patch agent (retrieval ceiling).
- **GM-progressive:** graph-guided retrieval feeds the retrieved file set to the patch agent.
- **RAG-progressive:** RAG retrieval feeds the retrieved file set to the patch agent.
- **None** (cold-start): the patch agent receives only the issue text with no retrieval context.

The patch agent uses `gemini-3-flash-preview` with a single generation attempt per instance (`patch_max_turns=1`), a 65,536-token output budget, a 200K-character file context limit, and a 480-second wall-clock cap per instance (`instance_wall_clock_cap_s=480`). Patches are evaluated using the official SWE-bench Docker harness for execution-based pass/fail assessment. Results are explicitly labeled as a pilot study: single run per method, no repeats.

Cost accounting disclosure. Frozen GM and RAG runs were produced under a dual-build pipeline in which both graph and RAG indices were built for GM and RAG methods respectively. We therefore report two cost views: *method-accounted* (design intent: GM pays only for graph build; RAG pays only for RAG index build) and *as-run-consumed* (actual API spend including the dual-build overhead). A split-build fix is implemented in `run_patch.py` for future runs; the as-run view captures the actual cost incurred during this pilot.

Timeout accounting. Instances that exceed the 480-second wall-clock cap are included in resolved-rate denominators (all-in primary metric). A timeout-excluded sensitivity analysis is reported alongside to separate method quality from infrastructure-imposed censoring.

5 Results

We present results organized by retrieval quality (§5.1), per-repository analysis (§5.2), amortization comparison (§5.3), cost analysis (§5.4), cross-benchmark validation (§5.5), and a patching pilot (§5.6).

Results marked *pending* indicate experiment cells that have not yet completed the readiness gates defined in §4.6. Completed cells report means over ≥ 3 repeats with bootstrap 95% CIs.

5.1 Main Retrieval Quality

Table 3 presents the cross-repository macro F1 for all seven methods under the strict commit-fidelity track.

From the three completed SWE-bench repositories:

- Graph methods win on Flask (`gm_baseline`: 0.809), while RAG methods win on Requests and Pytest (`rag_progressive`: 0.733 and 0.602 respectively).
- `gm_deterministic` achieves competitive F1 (0.679, 0.520, 0.473) with zero LLM calls at query time — all cost is setup embedding and lightweight query embedding.
- Non-LLM baselines (`raw_rag_*`) are not competitive, confirming that LLM-guided or graph-navigated retrieval provides substantial value.
- Cross-repo macro F1 across the three repos: RAG baseline 0.701, GM progressive 0.684, GM deterministic 0.557, RAG progressive 0.680, GM baseline 0.605.

5.2 Per-Repository Paired Analysis (psf/requests)

The most complete experiment cell is `psf/requests` with $n = 10$ issues and repeated runs on both tracks. Table 4 shows the strict-track repeat means.

The paired F1 delta for GM-progressive minus RAG-progressive is +0.181 with bootstrap 95% CI [0.120, 0.233] on this repository. This indicates that GM-progressive is competitive with and often higher than RAG-progressive on `psf/requests`; cross-repo results are mixed (Table 3), consistent with the paper’s cost-efficiency thesis rather than a quality-superiority claim.

5.3 Strict vs. Same-Snapshot Amortization

Table 5 compares the two evaluation tracks for the primary method pair.

Key observations:

- In the strict track, `commit_repeat_ratio` = 0.0 and `cache_hit_rate` = 0.0: every issue requires a fresh index build.
- In the same-snapshot track, `commit_repeat_ratio` = 0.9 and `cache_hit_rate` = 0.9: the graph is built once and reused.
- GM-progressive F1 improves from 0.467 (strict) to 0.544 (same-snapshot), suggesting that index stability benefits graph navigation.
- GM-deterministic same-snapshot cost is 10,745 tokens total vs. 109,284 strict—a 10 $\times$ reduction from amortization—while F1 improves from 0.520 to 0.603.

Method	SWE-bench repos				SWE-PolyBench repos		
	Flask	Requests	Pytest	FastAPI	yt-dlp	LangChain	Keras
gm_deterministic*	0.679	0.520	0.473	0.407 [†]	0.297 [‡]	0.452 [‡]	0.274 [‡]
gm_progressive	0.803	0.700	0.550	0.417 [†]	0.347 [‡]	0.783 [‡]	0.267 [‡]
gm_baseline	0.809	0.534	0.473	0.427 [†]	0.198 [‡]	0.312 [‡]	0.253 [‡]
rag_progressive	0.704	0.733	0.602	0.421 [†]	0.114 [‡]	0.366 [‡]	0.162 [‡]
rag_baseline	0.802	0.733	0.567	0.300 [†]	0.300 [‡]	0.433 [‡]	0.140 [‡]
raw_rag_function	0.340	0.256	0.242	0.255 [†]	0.145 [‡]	0.174 [‡]	0.179 [‡]
raw_rag_fixed	0.321	0.221	0.185	0.213 [†]	0.149 [‡]	0.132 [‡]	0.152 [‡]

Table 3: Per-repository mean F1 under the strict commit-fidelity track. Flask, Requests, and Pytest: means over ≥ 3 repeats for GM/RAG progressive/baseline; single run for deterministic and raw methods. [†]FastAPI: single run, SWE-bench ($n = 10$). [‡]yt-dlp ($n = 5$), LangChain ($n = 10$), Keras ($n = 10$): single run, SWE-PolyBench Verified. All single-run cells lack CIs; results are directional. *Flask, Requests, and Pytest **gm_deterministic** results use a tuned hyperparameter config (`configs/gm_deterministic_selected_v1.json`, selected via Flask-only tuning with Requests/Pytest as holdout); FastAPI, yt-dlp, LangChain, and Keras use default weights. Scoring formula weights differ substantially between configurations (see Appendix A.2).

Method	Mean F1	Runtime Tokens	Setup Tokens	Total Cost
GM (progressive)	0.467	43,283	107,557	150,840
GM (baseline)	0.379	51,898	107,557	159,455
RAG (progressive)	0.286	15,587	414,082	429,669
RAG (baseline)	0.332	40,756	414,082	454,838

Table 4: Strict commit-fidelity track: means over 3 repeats (`psf/requests`, $n = 10$).

This illustrates the maximum cost benefit of world-model reuse with no LLM overhead.

- The same-snapshot GM-progressive minus RAG-progressive delta is +0.286 with 95% CI [0.219, 0.333].

Figure 1 visualizes the strict vs. same-snapshot cost reduction alongside F1 for each method.

5.4 Cost Analysis

Table 6 presents the token cost decomposition across the three completed SWE-bench repositories (30 issues total, single run).

Figure 2 plots the F1-vs-cost frontier across all seven methods for the three completed SWE-bench repositories.

GM methods achieve $\sim 3.2\times$ lower setup cost than RAG methods (701K vs. 2.27M tokens) due to embedding compact graph metadata rather than full code bodies. GM-progressive total cost is $0.42\times$ that of RAG-progressive despite $2.4\times$ higher runtime token usage, because the setup cost dominates the total. GM-deterministic is the most cost-efficient retrieval method at 706K tokens total ($0.30\times$ of RAG-progressive), with zero LLM runtime spend.

5.5 Cross-Benchmark Validation (SWE-PolyBench)

All single-run SWE-PolyBench experiments are now complete (Table 3, [‡] columns). FastAPI results are from SWE-bench ([†]). All cross-benchmark cells are directional (single run, no CIs).

FastAPI (SWE-bench, $n = 10$): GM-prog (0.417) and GM-base (0.427) are competitive with RAG-prog (0.421) at substantially lower setup cost (67K vs. 137K setup tokens).

yt-dlp (SWE-PolyBench, $n = 5$): GM-prog (0.347) leads GM-det (0.297) and outperforms RAG-prog (0.114). RAG-baseline is competitive (0.300).

LangChain (SWE-PolyBench, $n = 10$): GM-prog achieves the highest single-repo F1 across all evaluated repositories (0.783), substantially outperforming RAG-prog (0.366). This suggests graph-guided methods particularly benefit from structural connectivity in framework-scale codebases.

Keras (SWE-PolyBench, $n = 10$): All methods obtain lower F1 than on SWE-bench repos, with GM-det (0.274) slightly ahead of GM-prog (0.267). RAG-baseline (0.140) trails raw-RAG methods (0.179/0.152), suggesting Keras issues involve less lexically predictable file locations. These lower values are consistent across methods and likely reflect benchmark difficulty rather than a method-specific effect.

Track	Method	Mean F1	Runtime	Setup	Total Cost
Strict	GM (progressive)	0.467	43,283	107,557	150,840
Strict	GM (deterministic)	0.520	1,727	107,557	109,284
Strict	RAG (progressive)	0.286	15,587	414,082	429,669
Same-snapshot	GM (progressive)	0.544	42,136	9,018	51,154
Same-snapshot	GM (deterministic)	0.603	1,727	9,018	10,745
Same-snapshot	RAG (progressive)	0.259	15,345	36,736	52,081

Table 5: Track-separated means (**psf/requests**, $n = 10$; GM-progressive/RAG-progressive: means over ≥ 3 repeats; GM-deterministic: single run). In same-snapshot mode, setup costs collapse due to single index build. GM-deterministic uses zero LLM runtime tokens.

Method	Runtime Tokens	Setup Tokens	Total Cost
GM (deterministic)	5,404	701,054	706,458
GM (progressive)	297,690	701,054	998,744
GM (baseline)	333,893	701,054	1,034,947
RAG (progressive)	123,317	2,268,044	2,391,361
RAG (baseline)	354,930	2,268,044	2,622,974
Raw-func	12,769	2,268,044	2,280,813
Raw-fixed	12,769	1,478,791	1,491,560

Table 6: Token cost across 3 SWE-bench repos (Flask, Requests, Pytest; 30 issues total, single run per method). GM-deterministic has negligible runtime cost (no LLM calls at query time).

5.6 Patching Pilot (N=100, SWE-bench Verified)

To evaluate end-to-end issue resolution, we run a patching pilot on 100 instances from the SWE-bench Verified subset across 9 repositories (see §4.9). We report resolved rate (pass@1 under the SWE-bench Docker harness) and cost-per-resolved-issue. All results are labeled explicitly as exploratory: single run per method, no repeats, no CIs.

Table 7 shows per-repository resolved rates. Targeted reruns for sympy, sphinx-doc, and matplotlib (which had timeout confounding in the original runs) are complete; all numbers are final.

Table 8 presents the cost-per-resolved-issue (CPR) under two accounting views. The method-accounted view reflects design intent (GM builds graph only; RAG builds RAG index only). The as-run-consumed view reflects actual API spend during the current pilot runs, in which both graph and RAG indices were built for GM and RAG runs (dual-build confound, see §4.9).

Figure 3 visualizes the per-repository resolved rates across all four methods.

Figure 4 compares cost-per-resolved-issue under method-accounted and as-run-consumed accounting.

Key observations (all exploratory, single-run pilot):

- Oracle resolves 45% of instances, providing the model ceiling under gold-file context. GM-progressive at 43% is directionally close to oracle.
- GM-progressive resolves 5 more instances than RAG-

progressive (43 vs. 38 out of 100). The quality difference is not statistically significant (McNemar exact $p = 0.38$, $n_{\text{discordant}} = 21$). Under method-accounted cost, GM-progressive CPR is approximately $4.4\times$ lower than RAG-progressive, consistent with the retrieval cost advantage.

- The GM-RAG gap narrows from 5pp all-in to 3.8pp when instances that exceeded the 480s wall-clock cap are excluded (timeout sensitivity analysis). A majority (16/21) of discordant instances are timeout-confounded; targeted reruns for sympy, sphinx-doc, and matplotlib are complete and reflected in the final numbers above.
- Cold-start (none) resolves only 3% of instances, confirming that retrieval context provides substantial lift for the patch model.

6 Discussion

When graph methods win. Graph-guided retrieval shows its strongest advantage on repositories with well-defined structural dependencies—particularly Flask, where import chains and class hierarchies directly connect issue-relevant files. The graph’s typed edges enable multi-hop navigation that reaches structurally related files even when their text content has low embedding similarity to the issue description. The setup cost advantage is also most pronounced in these cases: the graph index embeds only structural metadata (names, signatures, docstrings),

Repository	Oracle	GM-prog	RAG-prog	None
astropy/astropy ($n = 12$)	0.500	0.500	0.417	0.000
matplotlib/matplotlib ($n = 12$)	0.167	0.250	0.083	0.083
pallets/flask ($n = 1$)	1.000	1.000	0.000	0.000
psf/requests ($n = 8$)	0.625	0.500	0.500	0.125
pydata/xarray ($n = 12$)	0.500	0.417	0.583	0.000
pytest-dev/pytest ($n = 19$)	0.368	0.526	0.421	0.053
scikit-learn ($n = 12$)	0.500	0.500	0.417	0.000
sphinx-doc/sphinx ($n = 12$)	0.333	0.167	0.250	0.000
sympy/sympy ($n = 12$)	0.667	0.500	0.417	0.000
Pooled ($N = 100$)	0.450	0.430	0.380	0.030

Table 7: Patching pilot resolved rates (pass@1) under SWE-bench Docker harness. All values final; targeted reruns for sympy, sphinx-doc, and matplotlib complete. Rows sorted alphabetically.

Method	Resolved (out of 100)	CPR (tokens/resolved)	
		Method-accounted	As-run-consumed
Oracle	45	33,594	33,594
GM-progressive	43	479,354	1,250,676
RAG-progressive	38	2,115,746	2,319,677
None (cold-start)	3	24,291	24,291

Table 8: Cost-per-resolved-issue (CPR) for the patching pilot. Method-accounted CPR reflects design intent (GM builds graph only; RAG builds RAG index only); as-run-consumed reflects actual API spend including the dual-build confound (B10). None CPR appears low due to trivially low token spend on 3 cold-start resolutions. All numbers final.

requiring roughly one-third the embedding tokens of a full code chunk index.

When RAG methods win. On Requests and Pytest, RAG methods outperform graph methods on retrieval quality. Both repositories contain issues whose resolution involves files that are semantically related to the issue text but not strongly connected via import or call edges in the graph. This suggests that graph navigation is less effective when the structural signal is weak or when the issue description uses natural language that maps better to code semantics than to structural topology. The progressive RAG agent’s ability to iteratively refine its search also helps in cases where the initial retrieval set misses relevant files.

Cost-quality frontier. The most striking finding is the separation between retrieval quality and total cost (Figure 2). GM-progressive and RAG-progressive achieve similar macro F1 across the three completed SWE-bench repositories (0.684 vs. 0.680), but GM-progressive costs $0.42\times$ the total tokens. This cost advantage arises almost entirely from lower setup cost ($0.31\times$), partially offset by higher runtime LLM usage ($2.4\times$). As the number of tasks per snapshot increases, the amortized cost gap widens further in favor of graph methods.

Deterministic vs. LLM-guided retrieval. The deterministic graph scorer (`gm_deterministic`) provides a zero-LLM-runtime baseline that is fully reproducible and auditable. Observed F1 (0.679, 0.520, 0.473 on Flask, Requests, Pytest) is competitive with progressive LLM-guided methods on structurally regular repositories at $0.71\times$ the total cost of GM-progressive, confirming that much of the retrieval signal is encoded structurally. LLM-guided agents retain an advantage when issues require adaptive multi-hop exploration that the fixed BFS cannot anticipate.

Amortization structure. The dual-track evaluation reveals that amortization opportunity is highly dependent on experimental design (Figure 1). Under strict commit fidelity, most SWE-bench issue sets have `commit_repeat_ratio` near zero for small n , eliminating any index reuse. Under same-snapshot settings, the graph setup cost drops by an order of magnitude ($\sim 12\times$ for `psf/requests`). This structural observation is important for practical deployment: graph-based retrieval is most cost-effective in scenarios where a repository snapshot serves multiple queries, such as issue triage, code review, or batch analysis.

Patching pilot findings. The $N=100$ patching pilot (Tables 7–8, Figure 3) shows that GM-progressive resolves

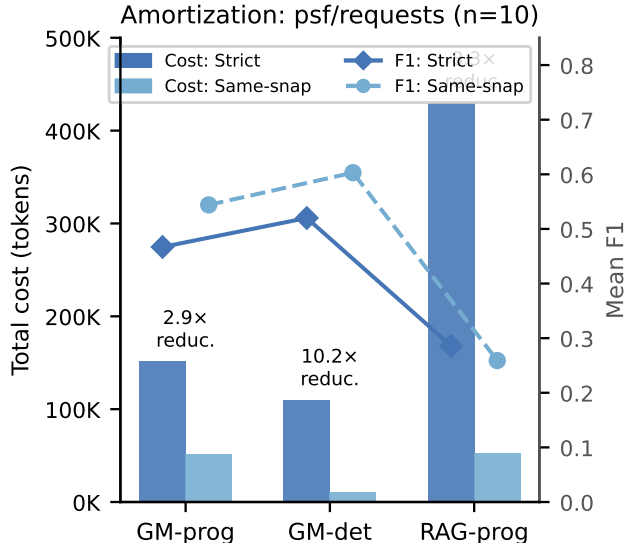


Figure 1: Strict vs. same-snapshot total token cost (bars) and mean F1 (lines) for GM-progressive, GM-deterministic, and RAG-progressive on `psf/requests` ($n = 10$). Cost reduction factors shown above bars.

43% of instances versus 38% for RAG-progressive and 3% for cold-start—directional evidence that retrieval context provides substantial lift and that graph-guided context is competitive with RAG in end-to-end resolution. These results are exploratory (single run, no CIs; McNemar $p = 0.38$). Three repositories (sympy, sphinx-doc, matplotlib) required targeted reruns to correct timeout confounding; those reruns are complete and reflected in all reported numbers. The method-accounted cost-per-resolved-issue gap (479K vs. 2.1M tokens, $\approx 4.4\times$; Figure 4) mirrors the retrieval setup cost advantage, suggesting the cost story holds end-to-end.

Limitations of current evidence. The results presented here are based on a limited number of repositories and issues. Key limitations include: (1) small sample sizes ($n = 10$ per repo for retrieval, single run for patching) that limit statistical power; (2) evaluation on a single LLM backend (`gemini-3-flash-preview`), leaving open the question of whether results transfer to other models; (3) the patching pilot uses a 480s wall-clock cap that creates timeout-induced right-censoring, particularly for RAG-progressive on complex repositories; and (4) Python-only graph construction, which constrains generalization to multi-language repositories.

7 Threats to Validity

Internal validity. LLM-guided retrieval methods are subject to API nondeterminism: the same input may produce different tool call sequences and file rankings across

runs. We mitigate this by running each experiment cell at least three times and reporting paired deltas with bootstrap 95% confidence intervals. Additionally, transient API errors (429 rate limits, 503 service unavailability) can cause individual issue evaluations to fail; we score such failures as $F1 = 0$ rather than excluding them, to avoid survivorship bias.

Issue sampling uses a deterministic first- n selection after split sweep and deduplication, which may introduce selection bias if early dataset instances are systematically different from later ones. Randomized sampling with multiple seeds would provide a stronger robustness check.

External validity. Our evaluation covers seven Python repositories across four application domains, but this remains a limited sample. Results may not generalize to repositories with different structural characteristics (e.g., heavily dynamic codebases, monorepos with non-standard layouts) or to other programming languages. The graph construction pipeline currently supports only Python via tree-sitter, limiting cross-language claims.

All experiments use a single LLM backend (`gemini-3-flash-preview`). Cost ratios and retrieval quality may differ with other model families, particularly models with different function-calling capabilities or token pricing.

Construct validity. We evaluate retrieval at the file level using precision, recall, and F1 against gold files extracted from patch diffs. This metric captures localization accuracy but does not measure whether the retrieved context is sufficient for successful patch generation. The patching pilot (§5.6) provides execution-based validation; results are exploratory at current sample sizes.

Timeout censoring (patching pilot). The 480-second wall-clock cap creates right-censoring on harder instances. Instances that time out are counted as unresolved in the primary all-in metric, which may bias both resolved rate and cost-per-resolved-issue if timeout rates differ by method. We report a timeout-excluded sensitivity analysis (§5.6) and conduct targeted reruns on the three most timeout-affected repositories. Experiments reflect researcher-local hardware and API quota constraints; production environments with higher quotas may reduce timeout pressure.

File-level granularity may also mask important within-file localization differences. Two methods may retrieve the same files but provide different amounts of useful context within those files. Node-level metrics, supported by SWE-PolyBench [15], would provide finer-grained evaluation.

Statistical validity. Sample sizes are small ($n = 5\text{--}10$ issues per repository), limiting the statistical power of per-repository comparisons. The cross-repository macro F1

GM cluster
(low cost)

RAG cluster
(high cost)

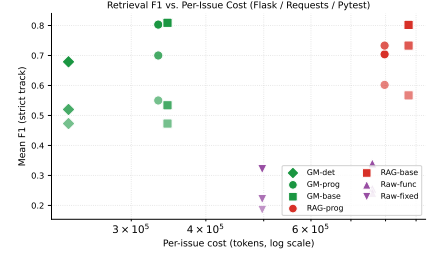


Figure 2: Retrieval F1 vs. per-issue cost (tokens, log scale) across seven methods for Flask, Requests, and Pytest. Graph methods (green) cluster at lower cost; RAG methods (red) at higher cost. Opacity encodes repository: Flask=100%, Requests=80%, Pytest=60%.

aggregates results from repositories of different difficulty, and the equal weighting may not reflect practical importance. The full retrieval matrix is now complete: SWE-bench repos (Flask, Requests, Pytest) with ≥ 3 repeats; FastAPI (single run, SWE-bench); yt-dlp, LangChain, and Keras (single run, SWE-PolyBench Verified). All single-run cells lack CIs. The patching pilot uses single runs across all methods with no repeat-based CIs; the McNemar test ($p = 0.38$) indicates the GM–RAG quality gap is not statistically significant at current sample sizes.

8 Conclusion

We presented GraphManager, a retrieval framework that leverages an AST-derived graph world model to provide cost-aware file localization for software issue resolution. The framework supports seven retrieval methods ranging from a deterministic graph scorer with zero LLM runtime cost to LLM-guided interactive agents, all evaluated under a dual-track protocol that separates strict commit-fidelity from same-snapshot amortization.

Our experiments on SWE-bench issues across multiple Python repositories show that graph-guided retrieval achieves competitive file-level F1 compared to strong RAG baselines while requiring substantially fewer total tokens. The graph index setup costs approximately $3\times$ fewer embedding tokens than dense code chunk indexing, and this advantage compounds in same-snapshot settings where the graph serves multiple retrieval tasks. A patching pilot on 100 SWE-bench Verified instances across 9 repositories provides directional evidence that this retrieval advantage translates to end-to-end issue resolution: GM-progressive resolves 43% of instances at approximately $4.4\times$ lower method-accounted cost per resolved issue than RAG-progressive (McNemar $p = 0.38$; quality difference not statistically significant). These pilot results are exploratory; targeted reruns for the three timeout-affected repositories are complete and reflected in these final numbers.

These results are preliminary: sample sizes are small and the patching pilot is a single-run study without confidence intervals. Expanding to larger instance sets and

adding repeat runs with confidence intervals are immediate priorities.

Looking forward, two extensions are particularly promising. First, incremental graph updates—recomputing only the affected subgraph when files change—could further reduce setup cost for evolving repositories. Second, extending the graph construction pipeline to additional languages via tree-sitter’s multi-language grammars would broaden the applicability of graph-guided retrieval to the multi-language codebases represented in benchmarks such as SWE-PolyBench [15].

References

- [1] Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. <https://github.com/tree-sitter/tree-sitter>, 2018.
- [2] Manideep Reddy Chinthareddy. Reliable graph-RAG for codebases: AST-derived graphs vs LLM-extracted knowledge graphs. *arXiv preprint arXiv:2601.08773*, 2026.
- [3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [4] Gemini Team, Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.
- [5] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, pages 11–15, 2008.
- [6] Nam Le Hai, Dung Nguyen Manh, and Nghi D. Q. Bui. RepoExec: Evaluate code generation with a repository-level executable benchmark. *arXiv*

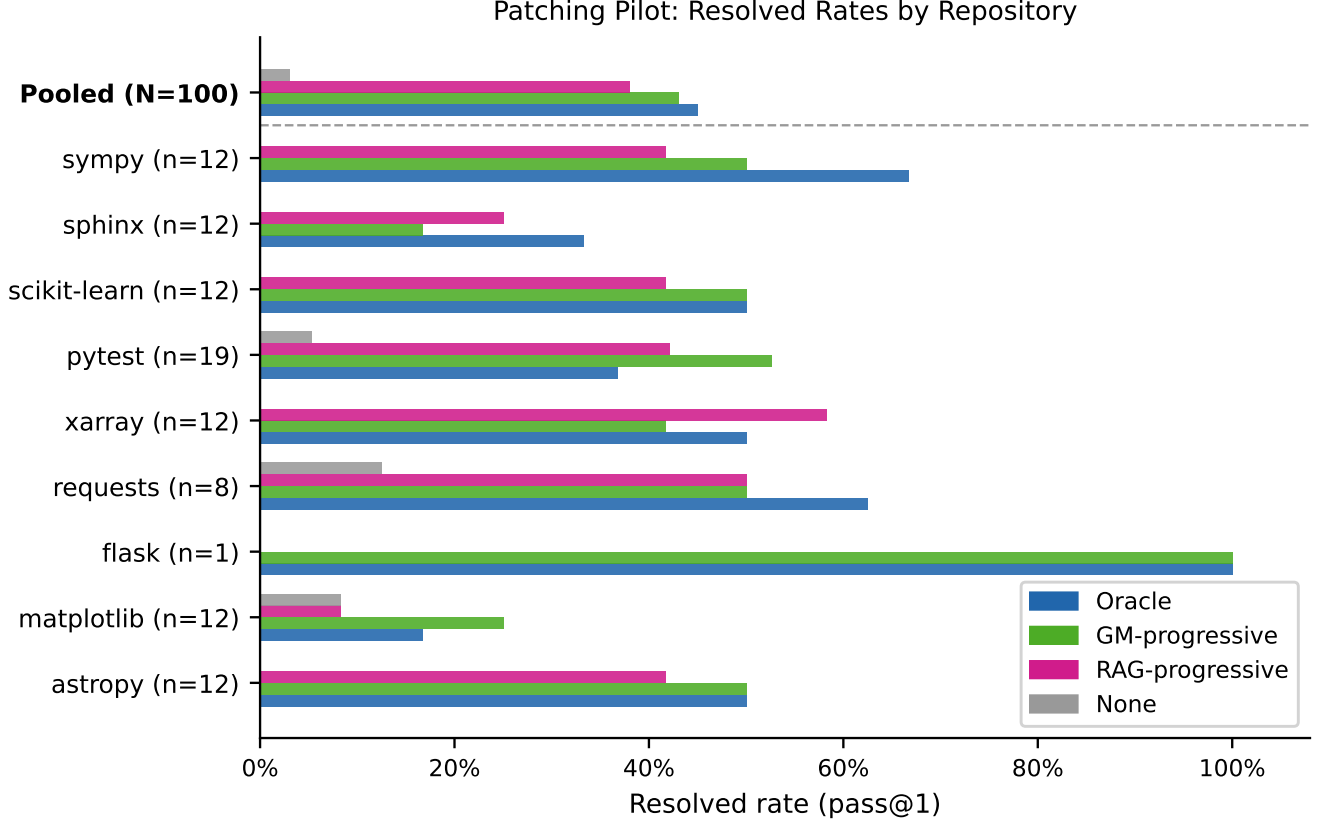


Figure 3: Patching pilot resolved rates (pass@1) per repository and pooled, across oracle, GM-progressive, RAG-progressive, and cold-start (none) methods. Pooled row (N=100) shown in bold at bottom.

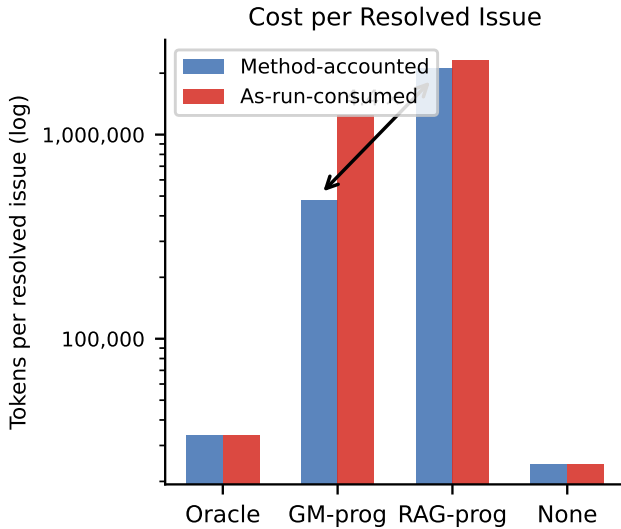


Figure 4: Cost-per-resolved-issue (CPR) in tokens, log scale. Method-accounted (blue) reflects design intent; as-run-consumed (red) reflects pilot dual-build cost. The $4.4\times$ ratio labels the GM vs. RAG method-accounted gap.

preprint arXiv:2406.11927, 2024. URL <https://arxiv.org/abs/2406.11927>.

- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [8] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [10] Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. In *International Conference*

on *Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2306.03091>.

- [11] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. GraphCoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.
- [12] Yang Liu, Li Zhang, Fang Liu, Zhuohang Wang, Donglin Wei, Zhishuo Yang, Kechi Zhang, Jia Li, and Lin Shi. RepoScope: Leveraging call chain-aware multi-view context for repository-level code generation. *arXiv preprint arXiv:2507.14791*, 2025.
- [13] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. *arXiv preprint arXiv:2410.14684*, 2024.
- [14] Huy Nhat Phan, Hoang Nhat Phan, Tien N. Nguyen, and Nghi D. Q. Bui. RepoHyper: Search-expand-refine on semantic graphs for repository-level code completion. In *Proceedings of the 2nd ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*, 2025. arXiv:2403.06095.
- [15] Muhammad Shihab Rashid et al. SWE-PolyBench: A multi-language benchmark for repository-level evaluation of coding agents. *arXiv preprint arXiv:2504.08703*, 2025. URL <https://arxiv.org/abs/2504.08703>.
- [16] vitali87. Code-graph-RAG. <https://github.com/vitali87/code-graph-rag>, 2025.
- [17] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [18] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- [19] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

A Appendix

A.1 Run Manifest Template

Each experiment cell is defined by a YAML manifest entry containing:

```
- repo: psf/requests
  domain: networking
  task_family: swe-bench
  dataset_name: SWE-bench/SWE-bench
  source_prefixes: [requests]
  n_issues: 10
  issue_set_id: issues_swebench_requests_v2_10
  instance_ids:
    - psf__requests-1142
    - psf__requests-1327
    - ...
  evaluation_track: strict_commit_fidelity
```

For same-snapshot runs, a `snapshot_commit` field (40-char SHA) pins the repository state alongside `evaluation_track: same_snapshot_amortized`.

Suite-level defaults include `manager_max_turns: 4`, `rag_max_turns: 4`, `repeats: 3`, and `seed: 17`.

A.2 Deterministic Scoring Formula Details

The five scoring components from Equation 1 are computed as follows:

- $S_{\text{sem}}(f)$: the maximum cosine similarity between the issue query embedding and any seed node belonging to file f .
- $S_{\text{graph}}(f)$: $\min(1, \log_2(1 + p) / \log_2(1 + P_{\text{max}}))$ where p is the number of unique graph paths reaching f and P_{max} is the maximum across all candidate files. Edge-type diversity provides a secondary boost.
- $S_{\text{conf}}(f)$: the mean confidence weight along paths to f , with edge weights: high = 1.0, medium = 0.6, low = 0.2.
- $S_{\text{hint}}(f)$: path-token precision—the fraction of lower-cased file/module name tokens that appear in the issue text ($|P \cap I| / |P|$ where P is the file-path token set and I is the issue-text token set). Note: this is precision over path tokens, not Jaccard similarity.
- $S_{\text{pen}}(f)$: penalty applied when the file is reachable only through low-confidence paths or is a known “hub” node (high in-degree relative to the graph).

Default coefficients: $w_{\text{sem}} = 0.35$, $w_{\text{graph}} = 0.30$, $w_{\text{conf}} = 0.20$, $w_{\text{hint}} = 0.10$, $w_{\text{pen}} = 0.05$.

A.3 Planned Ablations

The following ablation experiments are planned to isolate component contributions:

1. **Resolver confidence:** exclude low-confidence call edges during graph construction and measure the impact on retrieval quality.
2. **Scoring component isolation:**
 - Semantic only ($w_{\text{graph}} = w_{\text{conf}} = w_{\text{hint}} = w_{\text{pen}} = 0$).
 - Graph only ($w_{\text{sem}} = w_{\text{hint}} = 0$).
 - No confidence term ($w_{\text{conf}} = 0$).
 - No penalty term ($w_{\text{pen}} = 0$).
3. **Agent budget sweep:** vary `max_turns` from 2 to 8 for LLM-guided methods and measure quality-cost tradeoff.
4. **Seed count:** vary K_{seed} from 4 to 16 for the deterministic retriever.
5. **Expansion depth:** vary BFS depth D from 1 to 3.

A.4 Implementation Notes

Dependencies. Key Python packages: `tree-sitter` ($\geq 0.24.0$), `tree-sitter-python` ($\geq 0.23.0$), `networkx` (≥ 3.0), `faiss-cpu` ($\geq 1.7.4$), `google-genai` ($\geq 1.0.0$), `datasets` ($\geq 2.14.0$).

Execution. Experiments are run via `run_suite.py` with YAML configuration files. Single-repo experiments can also be run via `run_experiment.py` with CLI arguments. Results are written to timestamped directories under `results/runs/` and aggregated repeat sets under `results/repeat_sets/`.

Frozen artifacts. Report-ready numbers are generated by `research_report/generate_artifacts.py`, which reads raw results and produces a frozen bundle containing `manifest.json`, `summary_bundle.json`, and LaTeX table exports.

A.5 Figure Data Sources

All figures use only values that appear in the body tables; no additional data processing was performed. Data provenance:

- **Figure 3** (resolved rates): locked pilot numbers from Table 7; pooled $N=100$ totals as recorded in the session dev log (2026-02-22).
- **Figure 4** (CPR comparison): method-accounted and as-run-consumed values from Table 8.
- **Figure 2** (F1 vs. cost scatter): F1 from Table 3 (Flask/Requests/Pytest strict track); costs from Table 6 (30 issues total, single run), divided equally across 3 repos.
- **Figure 1** (amortization): cost and F1 from Table 5 (`psf/requests`, $n = 10$).

Figure scripts are in `research_report/figures/` and are reproducible from the frozen table values alone.

A.6 Attribution and Provenance

The graph construction pipeline in this project adapts resolver decomposition patterns from the open-source `code-graph-rag` project [16]. The adaptation is scoped to Python-first evaluation needs and does not import upstream product orchestration layers (Memgraph, MCP server, editing orchestration).

For detailed implementation provenance and external-credit mapping, see:

1. `docs/PROVENANCE_AND_CREDITS.md`
2. `docs/VITALI_PARITY_AUDIT.md`